



郑州大学
Zhengzhou University

第二单元 分组密码

郑州大学网络空间安全学院

王志华

王志华



第2单元 分组密码

提纲

CONTENTS

2.1 分组密码概述

2.2 分组密码分析

2.3 DES

2.4 AES

2.5 SM4

2.6 祖冲之密码

2.7 其他分组密码

2.8 分组密码工作模式

王华



一、我国的密码分级：

①核心密码：

用于保护党、政、军的核心机密。

②普通密码：

用于保护国家和事企业单位的低于核心机密而高于商业机密的密码信息。

③商用密码：

用于保护国家和事企业单位的非机密的敏感信息。

④个人密码：

用于保护个人的隐私信息。

前三种密码均由国家密码管理局统一管理。

王卫华



二、我国商业密码政策

① 统一领导：

国家密码管理局统一领导。

② 集中管理：

国家密码管理局办公室集中管理。

③ 定点研制：

研制只允许定点单位进行。

④ 专控经营：

经许可的单位才能经营。

⑤ 满足使用：

国内各单位都可申请使用。

2022/4/9



一、我国商用密码概况

(1) 密码的公开设计原则

密码的安全应仅依赖于对密钥的保密，不依赖于对算法的保密。

(2) 公开设计原则并不要求使用时公开所有的密码算法

核心密码不能公布算法；

核心密码的设计也要遵循公开设计原则。

(3) 商用密码应当公开算法

① 美国DES开创了公开商用密码算法的先例；

② 美国经历DES（公开）→EES（保密）→AES（公开）。

③ 欧洲也公布商用密码算法

2024



(4)我国的商用密码概况

- 我国在密码技术方面具有优势：

密码理论、密码分析

- 长期以来不公开密码算法，只提供密码芯片

少数专家设计，难免有疏漏；

难于标准化，不利于推广应用。

- 2006年2月我国公布了部分商用密码算法；

☆商用密码管理更科学化、与国际接轨；

☆将促进我国商用密码的发展。

2022



- 众所周知，为了保障商用密码的安全性，**国家商用密码管理办公室**制定了一系列密码标准，包括SM1（SCB2）、SM2、SM3、SM4、SM7、SM9、祖冲之密码算法（ZUC）等等。
 - SM1、SM4、SM7、祖冲之密码（ZUC）是对称算法；SM2、SM9是非对称算法；SM3是哈希算法。
 - SM1、SM7算法不公开，调用该算法时，需要通过加密芯片的接口进行调用。

王卫华



- SM4算法是用于WAPI的分组密码算法，是2006年我国国家密码管理局公布的用于无线局域网产品的分组对称密码算法，国内第一个商用密码算法，于2012年3月21日成为被批准的六项商码行业标准之一。
- SM4算法是分组密码算法，其中数据分组长度为128比特，密钥分组长度也为128比特。加密算法与密钥扩展算法都采用32轮迭代结构，以字节（8位）和字（32位）为单位进行数据处理。

王卫华



0、基本概况

SM4算法特点

分组	对合性	可逆性	算法安全
该算法的分组长度为128比特，密钥长度为128比特	加密算法和解密算法相同，算法相对简单	$SM4^{-1}(SM4(x_0, x_1, x_2, x_3)) = (x_0, x_1, x_2, x_3)$ 这个式子说明SM4算法是可逆的	计算过程中增加非线性变换，大大提高其算法的安全性

2022/4/9



0、基本概况

①分组密码:

数据分组长度=128位、密钥长度=128位

数据处理单位: 字节 (8位), 字 (32位)

②密码算法结构:

基本轮函数加迭代

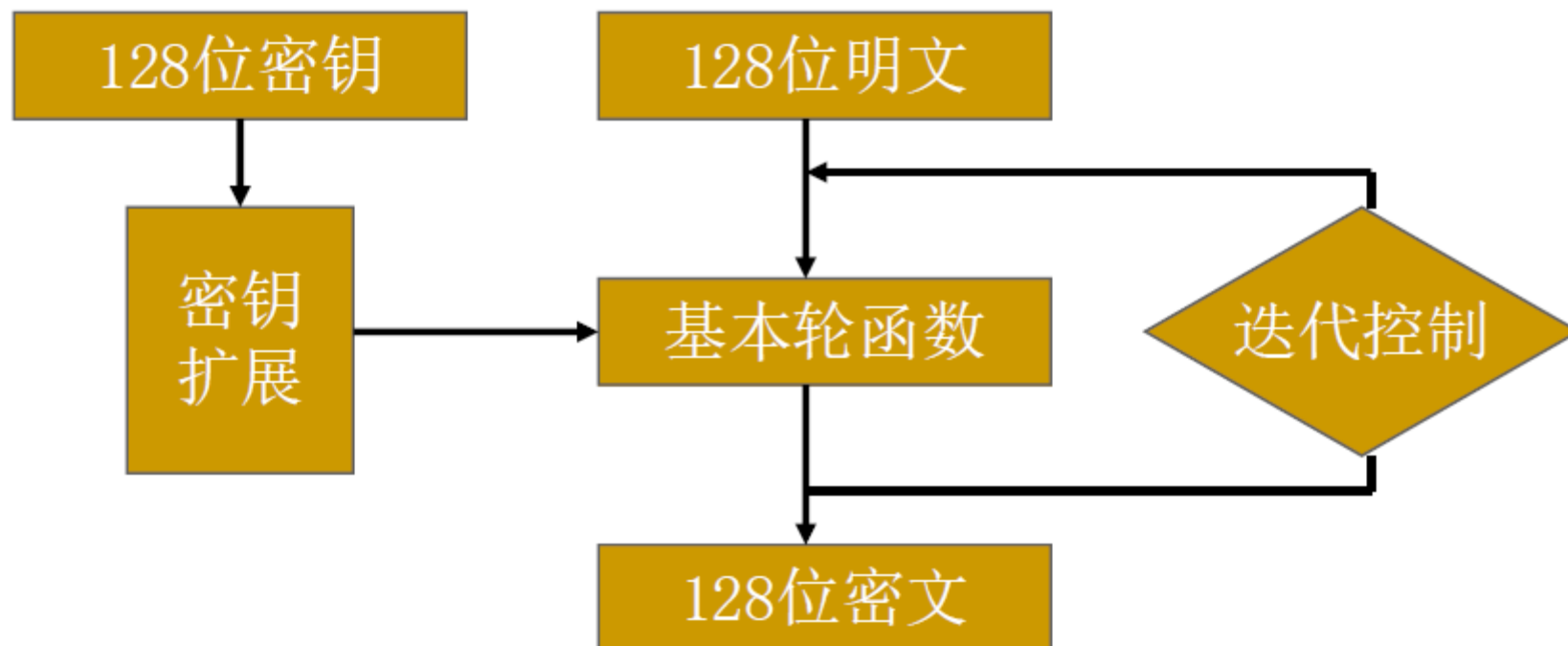
解密算法与加密算法相同

王卫华



0、基本概况

密码算法结构：



2022/4/9

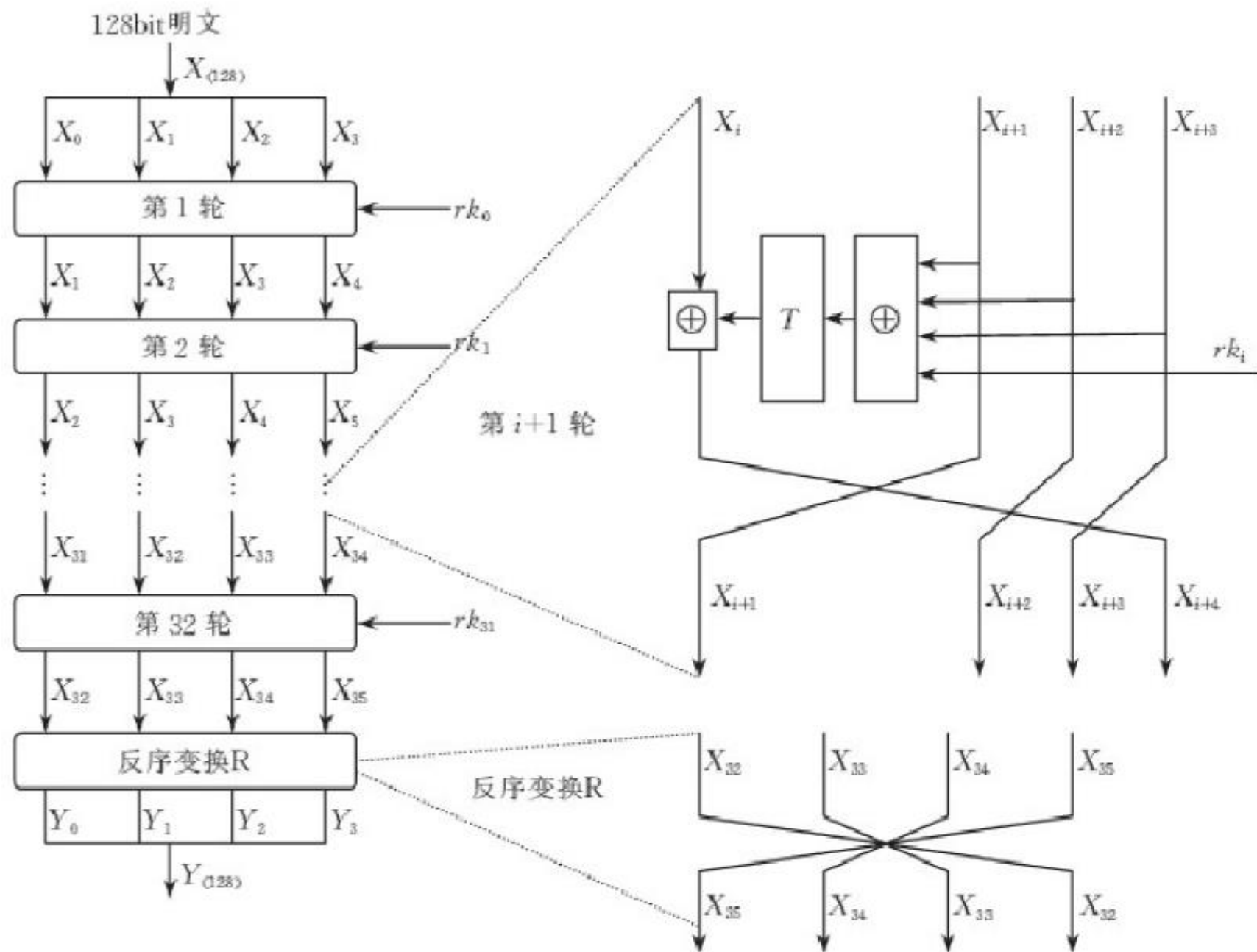


图 1 SMS4 加密算法整体结构



0、基本概况

- 非线性变换中所使用的**S**盒是一个具有很好密码学特性的、由**8**比特输入产生**8**比特输出的置换，在设计原理上，**SMS4**比**AES**的**S**盒设计多了一个仿射变换。
- **SMS4**有很高的灵活性，所采用的**S**盒可以灵活地被替换，以应对突发性的安全威胁。算法的**32**轮迭代采用串行处理，这与**AES**中每轮使用代换和混淆并行地处理整个分组有很大不同。
- 、它的解密算法与加密算法的结构相同，只是轮密钥的使用顺序相反，解密轮密钥是加密轮密钥的逆序。

2022/4/9



1、基本运算

- SM4密码算法的基本运算有模2加和循环移位。
 - ① 模2加：记为 $\oplus$ ，为32位逐比特异或运算。
 - ② 循环移位： $\lll i$ ，把32位字循环左移*i*位。

王卫华

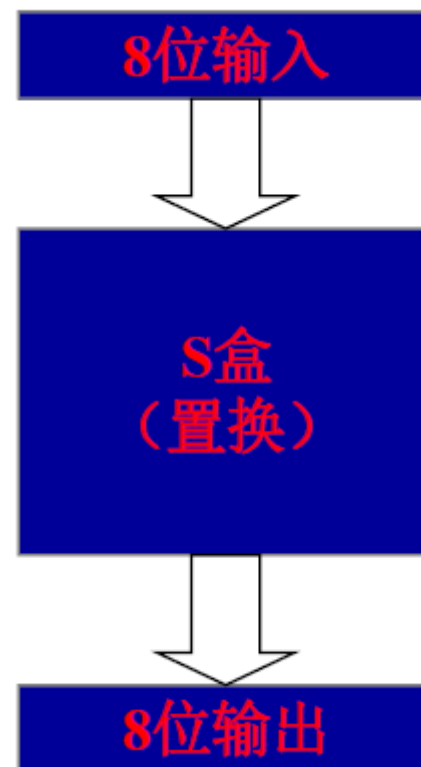


2、基本密码部件-S盒

- S盒是以字节为单位的非线性替换，其密码学作用是混淆，它的输入和输出都是8位的字节。
- 设输入字节为 a ,输出字节为 b , 则S盒的运算可表示为：

$$b = S(a)$$

- 本质上， S盒是8位的非线性置换。



2022/4/9



2、基本密码部件-S盒

- S盒的替换规则。

S盒的置换：

以输入的前半字节为行号，后半字节为列号，行列交叉点处的数据即为输出。

- 例如输入为EF，则输出为第E行与第F列交叉处的值84，即 $S(EF) = 84$ 。

王华



2、基本密码部件-S盒

		y															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	d6	90	e9	fe	cc	e1	3d	b7	16	b6	14	c2	28	fb	2c	05
	1	2b	67	9a	76	2a	be	04	c3	aa	44	13	26	49	86	06	99
	2	9c	42	50	f4	91	ef	98	7a	33	54	0b	43	ed	cf	ac	62
	3	e4	b3	1c	a9	c9	08	e8	95	80	df	94	fa	75	8f	3f	a6
	4	47	07	a7	fc	f3	73	17	ba	83	59	3c	19	e6	85	4f	a8
	5	68	6b	81	b2	71	64	da	8b	F8	eb	0f	4b	70	56	9d	35
	6	1e	24	0e	5e	63	58	d1	a2	25	22	7c	3b	01	21	78	87
	7	d4	00	46	57	9f	d3	27	52	4c	36	02	e7	a0	c4	c8	9e
	8	ea	bf	8a	d2	40	c7	38	b5	a3	f7	f2	ce	f9	61	15	a1
	9	e0	ae	5d	a4	9b	34	1a	55	ad	93	32	30	f5	8c	b1	e3
	a	1d	f6	e2	2e	82	66	ca	60	c0	29	23	ab	0d	53	4e	6f
	b	d5	db	37	45	de	fd	8e	2f	03	ff	6a	72	6d	6c	5b	51
	c	8d	1b	af	92	bb	dd	bc	7f	11	d9	5c	41	1f	10	5a	d8
	d	0a	c1	31	88	a5	cd	7b	bd	2d	74	d0	12	b8	e5	b4	b0
	e	89	69	97	4a	0c	96	77	7e	65	b9	f1	09	c5	6e	c6	84
	f	18	f0	7d	ec	3a	dc	4d	20	79	ee	5f	3e	D7	cb	39	48

2022/4/9



2、基本密码部件-非线性变换 τ

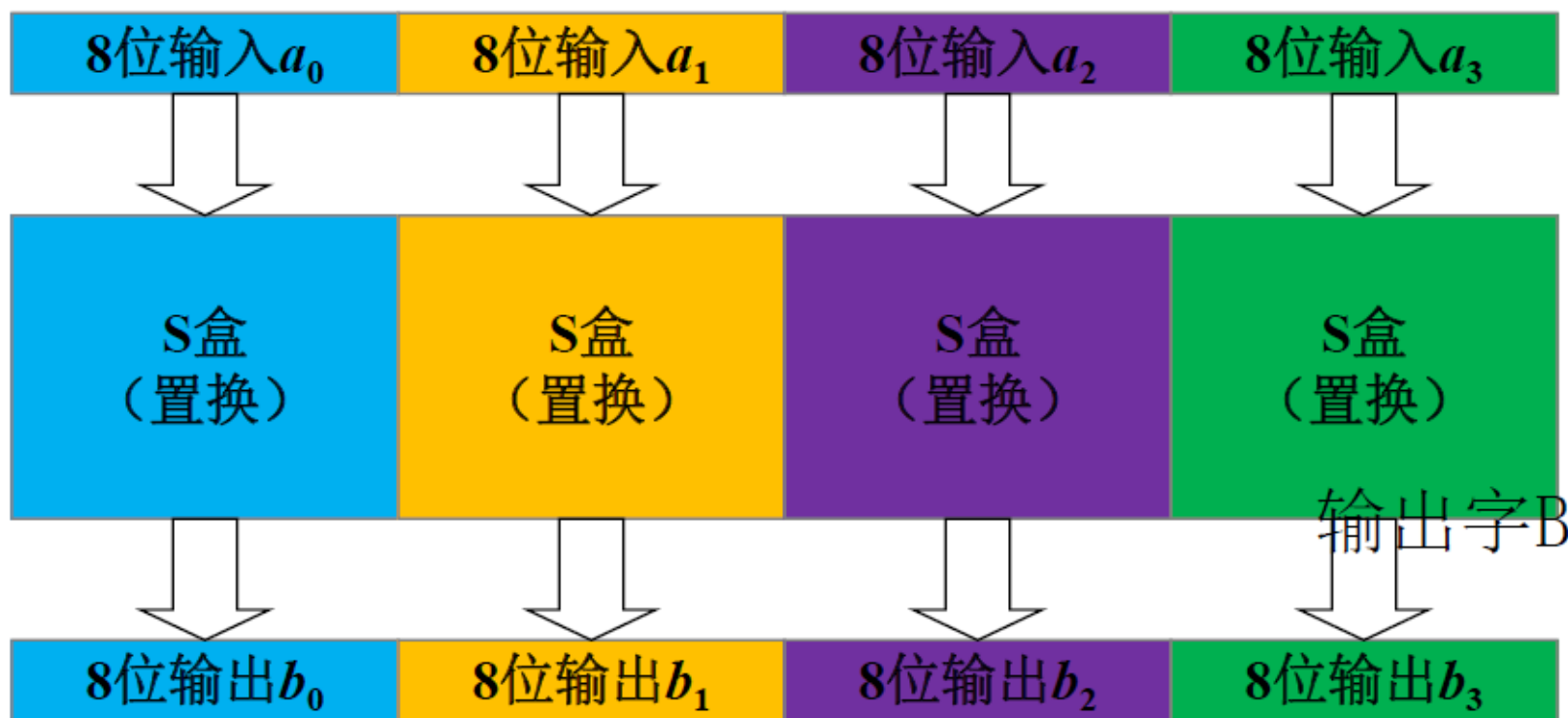
- 非线性变换 τ 是以字为单位的非线性替换，它由4个S盒并置构成。32位字的非线性变换
- 设输入为 $A = (a_0, a_1, a_2, a_3)$ (4个32位的字)，输出为
 $B = (b_0, b_1, b_2, b_3)$ (4个32位的字)，则
$$B = (b_0, b_1, b_2, b_3) = \tau(A) = (S(a_0), S(a_1), S(a_2), S(a_3))$$

2022/4/9



2、基本密码部件-非线性变换 τ

输入字 A



2022/4/9



2、基本密码部件-线性变换部件 L

- 线性变换部件 L 是以字为处理单位的线性变换，其输入输出都是32位的字，它的密码学作用是扩散。
- 设 L 的输入为字 B ，输出为字 C ，则

$$C = L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24)$$

王华



2、基本密码部件-合成变换 T

- 合成变换 T 由非线性变换 τ 和线性变换 L 复合而成，
数据处理的单位是字。设输入为字 X ，则先对 X 进行
非线性 τ 变换， 再进行线性 L 变换。记为：

$$T(X) = L(\tau(X))$$

- 由于合成变换 是非线性变换 和线性变换 的复合，
所以它综合起到混淆和扩散的作用， 从而可提高密
码的安全性。

2022/4/9



3、轮函数

☆输入数据： (X_0, X_1, X_2, X_3) ， 128位， 四个32位字。

☆输入轮密钥： rk ， 32位字。

☆输出数据： 32位字。

☆轮函数F：

$$F(X_0, X_1, X_2, X_3, rk) \\ = \underline{X_0 \oplus T(X_1 \oplus X_2 \oplus X_3 \oplus rk)}$$


2022/4/9



3、轮函数

轮函数由上述基本密码部件构成。设轮函数 F 的输入为4个32位字 (X_0, X_1, X_2, X_3) ，共128位，轮密钥为一个32位的字 rk 。输出也是一个32位的字，由下式给出：

$$F(X_0, X_1, X_2, X_3, rk) = X_0 \oplus T(X_1 \oplus X_2 \oplus X_3 \oplus rk)$$

根据式 (3-3)，有

$$F(X_0, X_1, X_2, X_3, rk) = X_0 \oplus L(\tau(X_1 \oplus X_2 \oplus X_3 \oplus rk))$$

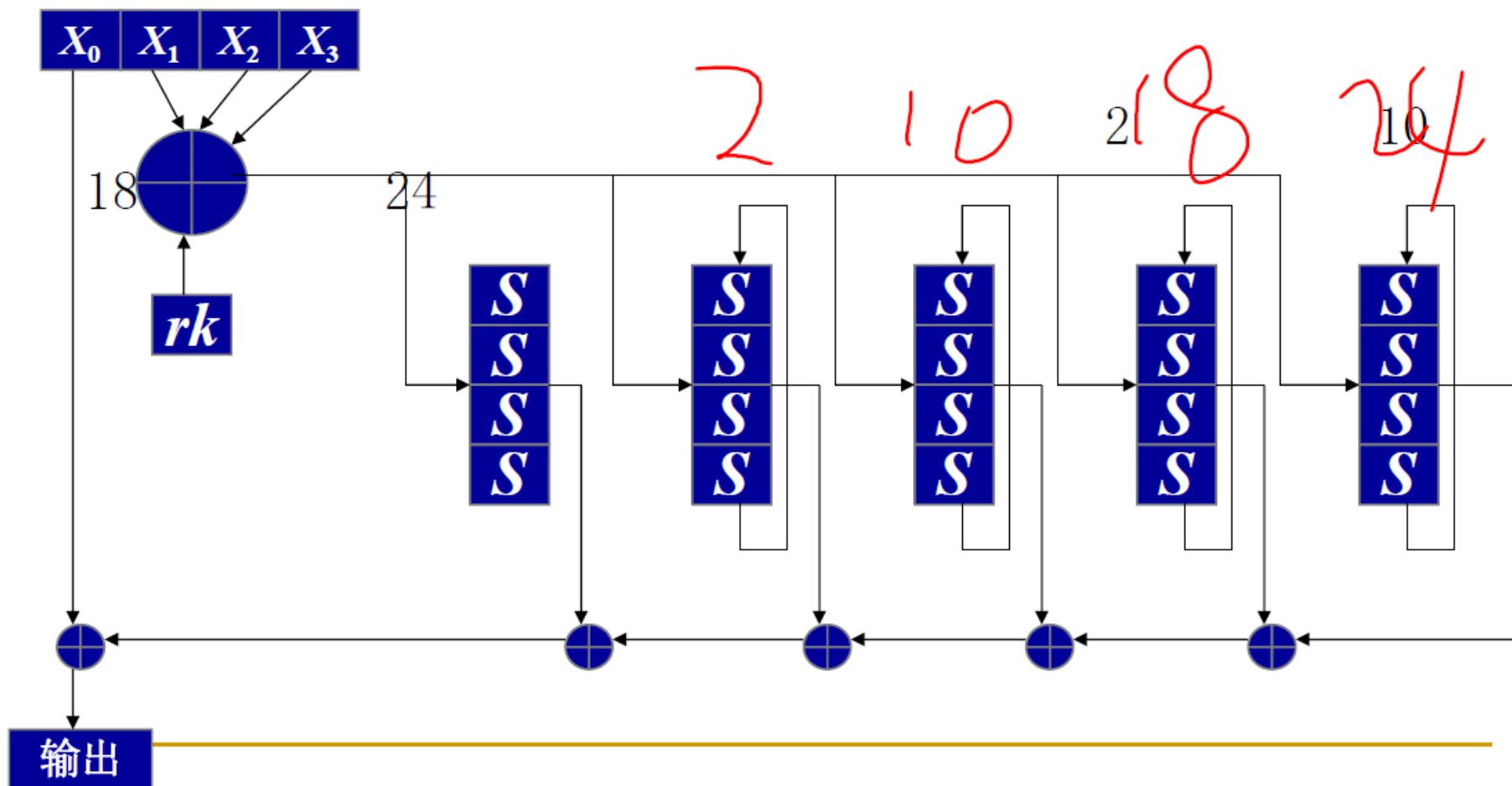
记 $B = (X_1 \oplus X_2 \oplus X_3 \oplus rk)$ ，根据式 (3-1) 和式 (3-2)，有

$$\begin{aligned} &F(X_0, X_1, X_2, X_3, rk) \\ &= X_0 \oplus [S(B)] \oplus [S(B) \lll 2] \oplus [S(B) \lll 10] \oplus [S(B) \lll 18] \oplus [S(B) \lll 24] \end{aligned}$$

2022/4/9



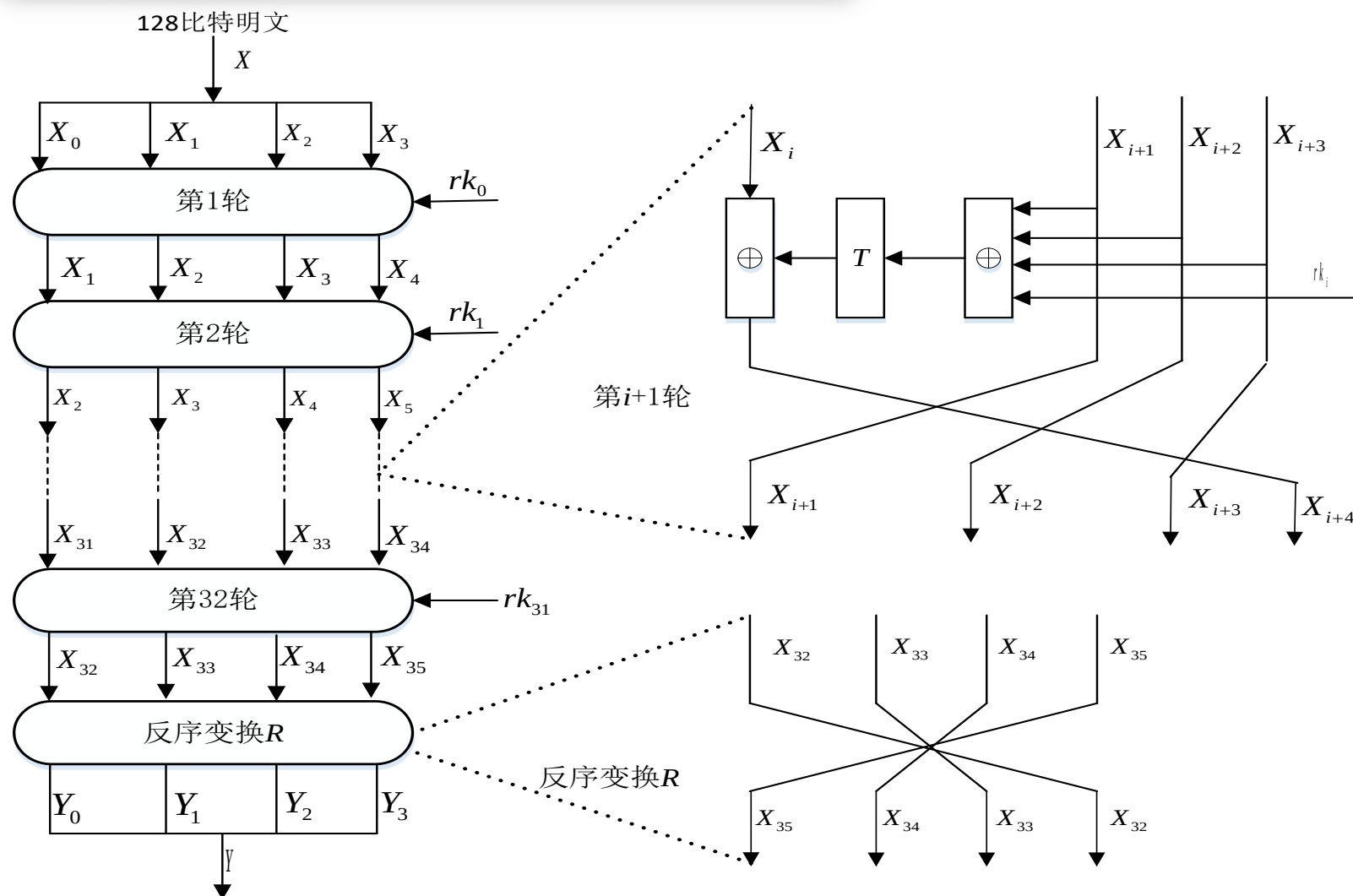
3、轮函数



2022/4/9



3、轮函数



2022/4/9



4、加密算法

加密算法采用32轮迭代结构，每轮使用一个轮密钥。

设输入的明文为四个字 (X_0, X_1, X_2, X_3) (128比特长)，输入的轮密钥为 rk_i ($i = 0, 1, \dots, 31$)，共32个字。输出的密文为四个字 (Y_0, Y_1, Y_2, Y_3) (128比特长)。加密算法可描述如下：

$$\begin{aligned} X_{i+4} &= F(X_i, X_{i+1}, X_{i+2}, X_{i+3}, rk_i) \\ &= X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i) \quad (i = 0, 1, \dots, 31) \end{aligned}$$

为了与解密算法需要的顺序一致，同时也与人们的习惯顺序一致，在加密算法之后还需要一个反序处理 R ：

(3-4)

$$(Y_0, Y_1, Y_2, Y_3) = (X_{35}, X_{34}, X_{33}, X_{32}) = R(X_{32}, X_{33}, X_{34}, X_{35})$$

2022/4/9



5、解密算法

解密算法与加密算法相同，只是轮密钥的使用顺序相反，解密轮密钥是加密轮密钥的逆序。

算法的输入为密文 (Y_0, Y_1, Y_2, Y_3) 和轮密钥 $rk_i (i = 31, 30, \dots, 1, 0)$ ，输出为明文 (X_0, X_1, X_2, X_3) 。根据式 $(Y_0, Y_1, Y_2, Y_3) = (X_{35}, X_{34}, X_{33}, X_{32})$ 为了便于与加密算法对照，解密算法中仍然用 X_i 表示密文。于是可得到如下的解密算法。

解密算法：

$$\begin{aligned} X_i &= F(X_{i+4}, X_{i+3}, X_{i+2}, X_{i+1}, rk_i) \\ &= X_{i+4} \oplus T(X_{i+3} \oplus X_{i+2} \oplus X_{i+1} \oplus rk_i) \quad (i = 31, 30, \dots, 1, 0) \end{aligned}$$

与加密算法之后需要一个反序处理同样的道理，在解密算法之后也需要一个反序处理 R ：

$$(X_0, X_1, X_2, X_3) = R(X_3, X_2, X_1, X_0)$$



6、密钥扩展算法

- 输入加密密钥: $MK = (MK_0, MK_1, MK_2, MK_3)$

- 输出轮密钥: rk_i , $i=0, 1, \dots, 30, 31$

- 中间数据: K_i , $i=0, 1, \dots, 34, 35$

- 密钥扩展算法:

① $(K_0, K_1, K_2, K_3) = (MK_0 \oplus FK_0, MK_1 \oplus FK_1, MK_2 \oplus FK_2, MK_3 \oplus FK_3)$

② For $i=0, 1, \dots, 30, 31$ Do

$$ik_i = K_{i+4} = K_i \oplus T' (K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)$$

- 说明: T' 变换与加密算法轮函数中的 T 基本相同, 只将其中的线性变换 L 修改为以下: L'

$$L' (B) = B \oplus (B \lll 13) \oplus (B \lll 23)$$

2022/4/9



6、密钥扩展算法

①常数FK 在密钥扩展中使用一些常数

$$\underline{FK_0}=(A3B1BAC6) \quad FK_1=(56AA3350)$$

$$FK_2=(677D9197) \quad FK_3=(B27022DC)$$

②固定参数CK

● 32 个固定参数 Ck_i , $i=0, 1, 2\cdots 31$

00070e15, 1c232a31, 383f464d, 545b6269,
70777e85, 8c939aa1, a8afb6bd, c4cbd2d9,
e0e7eef5, fc030a11, 181f262d, 343b4249,
50575e65, 6c737a81, 888f969d, a4abb2b9,
c0c7ced5, dce3eaf1, f8ff060d, 141b2229,
30373e45, 4c535a61, 686f767d, 848b9299,
a0a7aeb5, bcc3cad1, d8dfe6ed, f4fb0209,
10171e25, 2c333a41, 484f565d, 646b7279

产生规则: $Ck_{i,j} = (4i+j) \times 7 \pmod{256}$, $i=0, 1, 2\cdots 31, j=0, 1, \cdots 3$ 。

2022/4/9



6、密钥扩展算法

SM4算法加密时输入128位的密钥，采用32轮迭代结构，每一轮使用一个32位的轮密钥，共使用32个轮密钥。使用密钥扩展算法，从加密密钥产生出32个轮密钥。

(1) 常数 FK

在密钥扩展中使用如下的

常数: $FK_0 = (A3B1BAC6)$ $FK_1 = (56AA3350)$

$FK_3 = (B27022DC)$ $FK_2 = (677D9197)$

(2) 固定参数

共使用32个固定参数 CK_i ，每个 CK_i 是一个字，其产生规则如下：

设 $ck_{i,j}$ 为 CK_i 的第 j 字节 ($i = 0, 1, \dots, 31; j = 0, 1, 2, 3$)，即 $CK_i = (ck_{i,0}, ck_{i,1}, ck_{i,2}, ck_{i,3})$ ，则 $ck_{i,j} = (4i + j) \times 7 \pmod{256}$

2022/4/9



6、密钥扩展算法

这32个固定参数如下（16进制）：

00070E15,	1C232A31,	383F464D,	545B6269,
70777E85,	8C939AA1,	A8AFB6BD,	C4CBD2D9,
E0E7EEF5,	FC030A11,	181F262D,	343B4249,
50575E65,	6C737A81,	888F969D,	A4ABB2B9,
C0C7CED5,	DCE3EAF1,	F8FF060D,	141B2229,
30373E45,	4C535A61,	686F767D,	848B9299,
A0A7AEB5,	BCC3CAD1,	D8DFE6ED,	F4FB0209,
10171E25,	2C333A41,	484F565D,	646B7279

王华



6、密钥扩展算法

密钥扩展算法

设输入的加密密钥为 $MK = (MK_0, MK_1, MK_2, MK_3)$ ，输出轮密钥为 rk_i ($i = 0, 1, \dots, 30, 31$)，密钥扩展算法可描述如下，其中 K_i ($i = 0, 1, \dots, 35$) 为中间数据：

- $(K_0, K_1, K_2, K_3) = (MK_0 \oplus FK_0, MK_1 \oplus FK_1, MK_2 \oplus FK_2, MK_3 \oplus FK_3)$
- For $i = 0, 1, \dots, 31$ Do

$$rk_i = K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)$$

其中的变换 T' 与加密算法轮函数中的 T 基本相同，只将其中的线性变化 L 修改为以下的 L' ：

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23)$$

密钥扩展算法的结构与加密算法的结构类似，也是采用了32轮的迭代处理。

王华



SM4的安全性

- ①国家专业机构设计。算法简洁，以字和字节为处理单位，对合运算，符合当今分组密码主流。
- ②专业机构进行了密码分析，因此是安全的。
- ③民间学者对21轮SMS4进行了差分密码分析。
- ④尚需经过实践检验。

王卫华



SMS4的安全性

SMS4 密码算法的差分故障攻击

张 蕾 吴文玲

(中国科学院软件研究所信息安全国家重点实验室 北京 100080)

(中国科学院研究生院信息安全国家重点实验室 北京 100049)

摘 要 SMS4 是用于 WAPI 的分组密码算法,是国内官方公布的第一个商用密码算法.由于公布时间不长,关于它的安全性研究尚没有公开结果发表.该文研究 SMS4 密码算法对差分故障攻击的安全性.攻击采用面向字节的随机故障模型,并且结合了差分分析技术.该攻击方法理论上仅需要 32 个错误密文就可以完全恢复出 SMS4 的 128 比特种子密钥.因为实际中故障发生的字节位置是不可能完全平均的,所以实际攻击所需错误密文数将略大于理论值;文中的实验结果也验证了这一事实,恢复 SMS4 的 128bit 种子密钥平均大约需要 47 个错误密文.文章结果显示 SMS4 对差分故障攻击是脆弱的.为了避免这类攻击,建议用户对加密设备进行保护,阻止攻击者对其进行故障诱导.

吴文玲



第2单元 分组密码

提纲

CONTENTS

2.1 分组密码概述

2.2 分组密码分析

2.3 DES

2.4 AES

2.5 SM4

2.6 祖冲之密码

2.7 其他分组密码

2.8 分组密码工作模式

2.2.4

2.6 祖冲之密码



郑州大学
Zhengzhou University

- 祖冲之密码。

2022/4/9

3.8.1 算法中的符号及含义

数制表示

文中整数如果不加特殊说明都为十进制，如果有前缀“0x”则表示十六进制，如果有下标“2”则表示二进制。

例 整数 a 可以有以下不同数制表示形式。

$a = 1234567890$

十进制表示

$= 0x499602D2$

十六进制表示

$= 10010011001011000000010110100102$

二进制表示

数据位序

文中所有数据的最高位（或字节）在左边，最低位（或字节）在右边。如 $a = 10010011001011000000010110100100$ ， a 的最高位为其最左边一位1， a 的最低位为其最右边一位0。



3.8.1 算法中的符号及含义

运算符号表示

$+$	两个整数加
ab	两个整数 a 和 b 相乘
$=$	赋值运算
mod	整数取模
$\oplus$	整数间逐比特异或（模2加）
$\boxed{+}$	模 2^{32} 加
$a b$	串 a 和 b 级联
a_H	整数 a 的高（最左）16位
a_L	整数 a 的低（最右）16位
$a \lll k$	a 循环左移 k 位
$a \gg 1$	a 右移一位
$(a_1, a_2, \dots, a_n) \rightarrow (b_1, b_2, \dots, b_n)$	a_i 到 b_i 的并行赋值



3.8.1 算法中的符号及含义

例 $a = 0x1234$ $b = 0x5678$ $c = a \parallel b = 0x12345678$

例 $a = 100100110010110000001011010010_2$
 $a_H = 1001001100101100_2$ $a_L = 0000001011010010_2$

例 $a = 110010011001011000001011010010_2$
 $a \gg 1 = 1100100110010110000000101101001_2$

例 设 $a_1, a_2, \dots, a_{15}$ 都是整数 $(a_1, a_2, \dots, a_{15}) \rightarrow (b_1, b_2, \dots, b_{15})$
 $b_i = a_i (1 \leq i \leq 15)$



3.8.2 祖冲之密码的算法结构

算法从逻辑上分为上中下三层，见图3-25。上层是16级线性反馈移位寄存器（LFSR），中层是比特重组（BR- BitReconstruction），下层是非线性函数 F 。

线性反馈移位寄存器

线性反馈移位寄存器（LFSR）由16个31比特寄存器单元 $s_0, s_1, \dots, s_{15}$ 组成，每个单元在集合

$\{ 1, 2, 3, \dots, 2^{31} - 1 \}$ 中取值。

线性反馈移位寄存器的特征多项式是有限域 $GF(2^{31} - 1)$ 上的16次本原多项式

$$p(x) = x^{16} - 2^{15}x^{15} - 2^{17}x^{13} - 2^{21}x^{10} - 2^{20}x^4 - (2^8 + 1)$$

因此，其输出为有限域 $GF(2^{31} - 1)$ 上的 m 序列，具有良好的随机性。



3.8.2 祖冲之密码的算法结构

线性反馈移位寄存器的运行模式有两种：初始化模式和工作模式。

(1) 初始化模式

在初始化模式中，LFSR接收一个31比特字，是由非线性函数的32比特输出通过舍弃最低位比特得到，即 $w \gg 1$ 。计算过程如下：

LFSRWithInitialisationMode(u)

- ① $v = 2^{15}s_{15} + 2^{17}s_{13} + 2^{21}s_{10} + 2^{20}s_4 + (1 + 2^8)s_0 \bmod (2^{31} - 1)$ ；
- ② $s_{16} = (v + u) \bmod (2^{31} - 1)$ ；
- ③ 如果 $s_{16} = 0$ ，则置 $s_{16} = 2^{31} - 1$ ；
- ④ $(s_1, s_2, \dots, s_{15}, s_{16}) \rightarrow (s_0, s_1, \dots, s_{14}, s_{15})$ 。



3.8.2 祖冲之密码的算法结构

(2) 工作模式

在工作模式下，LFSR没有输入。其计算过程如下：

LFSRWithWorkMode()

$$\textcircled{1} s_{16} = 2^{15} s_{15} + 2^{17} s_{13} + 2^{21} s_{10} + 2^{20} s_4 + (1 + 2^8) s_0 \bmod (2^{31} - 1) ;$$

$$\textcircled{2} \text{如果 } s_{16} = 0, \text{ 则置 } s_{16} = 2^{31} - 1;$$

$$\textcircled{3} (s_1, s_2, \dots, s_{15}, s_{16}) \rightarrow (s_0, s_1, \dots, s_{14}, s_{15})。$$



3.8.2 祖冲之密码的算法结构

比较两种模式，差别在于初始化时需要引入由非线性函数输出的通过舍弃最低位比特得到，而工作模式不需要。目的在于，引入非线性函数的输出 F ，使线性反馈移位寄存器的状态随机化。LFSR的作用主要是为中层的比特重组（BR）提供随机性良好的输入驱动。

比特重组

比特重组从LFSR的寄存器单元中抽取128比特组成4个32比特字 X_0, X_1, X_2, X_3 ，其中前3个字将用于下层的非线性函数，第4个字参与密钥流的计算。



3.8.2 祖冲之密码的算法结构

比特重组

具体计算过程如下：

BitReconstruction()

$$1. X_0 = s_{15H} \parallel s_{14L}$$

$$2. X_1 = s_{11L} \parallel s_{9H}$$

$$3. X_2 = s_{7L} \parallel s_{5H}$$

$$4. X_3 = s_{2L} \parallel s_{0H}$$



3.8.2 祖冲之密码的算法结构

非线性函数 F

非线性函数 有2个32比特长的存储单元 R_1 和 R_2 ，其输入为来自上层比特重组的3个32比特字 X_0 、 X_1 、 X_2 ，输出为一个32比特字 W 。因此，非线性函数 F 是一个把96比特压缩为32比特的一个非线性压缩函数。

具体计算过程如下：

$$\begin{aligned}
 &F(X_0, X_1, X_2) \\
 &W = (X_0 \oplus R_1) \boxed{+} R_2 \\
 &W_1 = R_1 \boxed{+} X_1 \\
 &W_2 = R_2 \oplus X_2 \\
 &R_1 = S(L_1(W_{1L} \parallel W_{2H})) \\
 &R_2 = S(L_2(W_{2L} \parallel W_{1H}))
 \end{aligned}$$



3.8.2 祖冲之密码的算法结构

S盒: 32×32 (即输入长和输出长都为32比特) 的S盒 由4个并置的 S盒构成, 即

$$S = (S_0, S_1, S_2, S_3)$$

其中 $S_2 = S_0, S_3 = S_1$, 于是有

$$S = (S_0, S_1, S_0, S_1)$$

S_0 和 S_1 的定义分别如表3-12和3-13所示。

设 x 是 S_0 (或 S_1) 的8比特长输入, 将 x 写成2个16进制数 $x = h \| l$ 那么 (或) 的输出是表3-12 (或表3-13) 第 h 行和第 l 列交叉位置的16进制数。



3.8.2 祖冲之密码的算法结构

表3-13 S1盒

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	3E	72	5B	47	CA	E0	00	33	04	D1	54	98	09	B9	6D	CB
1	7B	1B	F9	32	AF	9D	6A	A5	B8	2D	FC	1D	08	53	03	90
2	4D	4E	84	99	E4	CE	D9	91	DD	B6	85	48	8B	29	6E	AC
3	CD	C1	F8	1E	73	43	69	C6	B5	BD	FD	39	63	20	D4	38
4	76	7D	B2	A7	CF	ED	57	C5	F3	2C	BB	14	21	06	55	9B
5	E3	EF	5E	31	4F	7F	5A	A4	0D	82	51	49	5F	BA	58	1C
6	4A	16	D5	17	A8	92	24	1F	8C	FF	D8	AE	2E	01	D3	AD
7	3B	4B	DA	46	EB	C9	DE	9A	8F	87	D7	3A	80	6F	2F	C8
8	B1	B4	37	F7	0A	22	13	28	7C	CC	3C	89	C7	C3	96	56
9	07	BF	7E	F0	0B	2B	97	52	35	41	79	61	A6	4C	10	FE
A	BC	26	95	88	8A	B0	A3	FB	C0	18	94	F2	E1	E5	E9	5D
B	D0	DC	11	66	64	5C	EC	59	42	75	12	F5	74	9C	AA	23
C	0E	86	AB	BE	2A	02	E7	67	E6	44	A2	6C	C2	93	9F	F1
D	F6	FA	36	D2	50	68	9E	62	71	15	3D	D6	40	C4	E2	0F
E	8E	83	77	6B	25	05	3F	0C	30	EA	70	B7	A1	E8	A9	65
F	8D	27	1A	DB	81	B3	A0	F4	45	7A	19	DF	EE	78	34	60



3.8.2 祖冲之密码的算法结构

表3-13 S2盒

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	55	C2	63	71	3B	C8	47	86	9F	3C	DA	5B	29	AA	FD	77
1	8C	C5	94	0C	A6	1A	13	00	E3	A8	16	72	40	F9	F8	42
2	44	26	68	96	81	D9	45	3E	10	76	C6	A7	8B	39	43	E1
3	3A	B5	56	2A	C0	6D	B3	05	22	66	BF	DC	0B	FA	62	48
4	DD	20	11	06	36	C9	C1	CF	F6	27	52	BB	69	F5	D4	87
5	7F	84	4C	D2	9C	57	A4	BC	4F	9A	DF	FE	D6	8D	7A	EB
6	2B	53	D8	5C	A1	14	17	FB	23	D5	7D	30	67	73	08	09
7	EE	B7	70	3F	61	B2	19	8E	4E	E5	4B	93	8F	5D	DB	A9
8	AD	F1	AE	2E	CB	0D	FC	F4	2D	46	6E	JD	97	E8	D1	E9
9	4D	37	A5	75	5E	83	9E	AB	82	9D	B9	1C	E0	CD	49	89
A	01	B6	BD	58	24	A2	5F	38	78	99	15	90	50	B8	95	E4
B	D0	91	C7	CE	ED	0F	B4	6F	A0	CC	F0	02	4A	79	C3	DE
C	A3	EF	EA	51	E6	6B	18	EC	1B	2C	80	F7	74	E7	FF	21
D	5A	6A	54	1E	41	31	92	35	C4	33	07	0A	BA	7E	0E	34
E	88	B1	98	7C	F3	3D	60	6C	7B	CA	D3	1F	32	65	04	28
F	64	BE	85	9B	2F	59	8A	D7	B0	25	AC	AF	12	03	E2	F2



3.8.2 祖冲之密码的算法结构

例： $S_0(0x12)=0xF9$, $S_1(0x34)=0xC0$

设 的输入、输出分别为 (32比特长) 和 (32比特长) ,
将 和 分别表示成4个字节 $X = x_0 \| x_1 \| x_2 \| x_3$, $Y = y_0 \| y_1 \| y_2 \| y_3$

例 设 $y_i = S_i(x_i), (i = 0, 1, 2, 3)$ 则

$$S_0(0x12)=0xF9, S_1(0x34)=0xC0$$

$$Y = S(X) = S_0(0x12)S_1(0x34)S_2(0x56)S_4(0x78) = 0xF9C05A4E$$

(2) 线性变换 L_1 和 L_2 为 32比特 线性变换，定义如下：

$$\begin{cases} L_1(X) = X \oplus (X \lll 2) \oplus (X \lll 10) \oplus (X \lll 18) \oplus (X \lll 24) \\ L_2(X) = X \oplus (X \lll 8) \oplus (X \lll 14) \oplus (X \lll 22) \oplus (X \lll 30) \end{cases}$$

其中符号 $\lll$ 表示把 a 循环左移 n 位。

a

a

n



3.8.2 祖冲之密码的算法结构

与式 (3-2) 相比可知, 式 (3-4) 中的 $\mathbf{L}$ 与SM4密码中的线性变换 $\mathbf{L}$ 相同。

在非线性函数 $\mathbf{F}$ 中采用非线性变换S盒的目的是提供混淆作用, 采用线性变换 $\mathbf{L}$ 的目的是提供扩散作用。正是混淆和扩散相互配合提高了密码的安全性。

非线性函数 $\mathbf{F}$ 输出的 $\mathbf{Y}$ 与比特重组 (BR) 输出的 $\mathbf{Z}$ 异或, 形成输出密钥序列 $\mathbf{K}$ 。

算法结构图3-25与第2章图2-6一致。



3.8.2 祖冲之密码的算法结构

密钥装载

密钥装载过程将128比特的初始密钥 和128比特的初始向量 扩展为16个31比特长的整数，作为LFSR寄存器单元 $s_0, s_1, \dots, s_{15}$ 的初始状态。

设 k 和 IV 分别为

$$k = k_0 \parallel k_1 \parallel \dots \parallel k_{15}$$

和

$$IV = iv_0 \parallel iv_1 \parallel \dots \parallel iv_{15}$$

其中： k_i 和 iv_i 均为8比特长字节， $0 \leq i \leq 15$ 密钥装入过程如下。



3.8.2 祖冲之密码的算法结构

1. 设 D 为240比特的常量，可按如下方式分成16个15比特的子串：

$$D = d_0 \parallel d_1 \parallel \cdots \parallel d_{15}$$

2. 对 $0 \leq i \leq 15$ 取 $s_i = k_i \parallel d_i \parallel iv_i$



3.8.2 祖冲之密码的算法结构

其中 d_i 的二进制表示为：

$$d_0 = 100010011010111_2$$

$$d_1 = 010011010111100_2$$

$$d_2 = 110001001101011_2$$

$$d_3 = 001001101011110_2$$

$$d_4 = 101011110001001_2$$

$$d_5 = 011010111100010_2$$

$$d_6 = 111000100110101_2$$

$$d_7 = 000100110101111_2$$

$$d_8 = 100110101111000_2$$

$$d_9 = 010111100010011_2$$

$$d_{10} = 110101111000100_2$$

$$d_{11} = 001101011110001_2$$

$$d_{12} = 101111000100110_2$$

$$d_{13} = 011110001001101_2$$

$$d_{14} = 111100010011010_2$$

$$d_{15} = 100011110101100_2$$



3.8.3 祖冲之密码的运行

算法的运行有两个阶段：初始化阶段和工作阶段

(1) 初始化阶段

调用密钥装载过程，将128比特的初始密钥 和128比特的初始向量 装入到 LFSR 寄存器单元变量 中， s_0, s_1 作为 s_{15} 的初态，并置非线性函数 中的32比特存储单元 和 全为0。 R_1, R_2

然后重复执行以下过程32次：

BitReconstruction()

$$W = F(X_0, X_1, X_2)$$

LFSRWithInitialisationMode (u)



3.8.3 祖冲之密码的运行

(2) 工作阶段

初始化阶段以后，执行工作阶段。

首先执行以下过程一次，并将 I 的输出 舍弃：

1. BitReconstruction()
2. $F(X_0, X_1, X_2)$
3. LFSRWithWorkMode()

然后进入密钥输出阶段，其中每进行一次循环，执行以下过程一次，输出一个32比特的密钥字：

1. BitReconstruction()
2. $Z = F(X_0, X_1, X_2) \oplus X_3$
3. LFSRWithWorkMode()



3.8.4 祖冲之密码的机密性算法 128-EEA3

基于祖冲之密码的机密性算法，主要用于4G移动通信中移动用户设备UE和无线网络控制设备RNC之间的无线链路上通信信令和数据的加解密。

算法的输入与输出

算法的输入参数见表3-14，输出参数见表3-15。

表3-15 ZUC机密性算法输出参数表

输出参数	比特长度	备注
C	$LENGTH$	输出比特流



3.8.4 祖冲之密码的机密性算法 128-EEA3

表3-14 ZUC机密性算法输入参数表

输入参数	比特长度	备注
COUNT	32	计数器
BEARER	5	承载层标识
DIRECTION	1	传输方向标识
CK	128	机密性密钥
LENGTH	32	明文消息的比特长度
M	LENGTH	明文消息的比特流



3.8.4 祖冲之密码的机密性算法 128-EEA3

算法工作流程

加解密算法如图3-26所示。

(1) 初始化

初始化是指根据机密性密钥 CK 以及其他输入参数（见表3-14）构造祖冲之算法的初始密钥 k 和初始向量 IV 。把 CK (128比特长) 和 k (128比特长) 分别表示为16个字节：

$$CK = CK[0] \parallel CK[1] \parallel CK[2] \parallel \cdots \parallel CK[15]$$

$$k = k[0] \parallel k[1] \parallel k[2] \parallel \cdots \parallel k[15]$$

$$k[i] = CK[i] \quad i = 0, 1, 2, \cdots, 15$$



3.8.4 祖冲之密码的机密性算法 128-EEA3

把计数器 $COUNT$ (32比特长) 表示为4个字节:

$$COUNT = COUNT[0] || COUNT[1] || COUNT[2] || COUNT[3]$$

把 IV (128比特长) 表示为16个字节:

$$\text{令} \left\{ \begin{array}{l} IV[0] = COUNT[0], IV[1] = COUNT[1], \\ IV[2] = COUNT[2], IV[3] = COUNT[3], \\ IV[4] = BEARER || DIRECTION || 00_2, \\ IV[5] = IV[6] = IV[7] = 00000000_2, \\ IV[8] = IV[0], IV[9] = IV[1], \\ IV[10] = IV[2], IV[11] = IV[3], \\ IV[12] = IV[4], IV[13] = IV[5], \\ IV[14] = IV[6], IV[15] = IV[7]. \end{array} \right.$$



3.8.4 祖冲之密码的机密性算法 128-EEA3

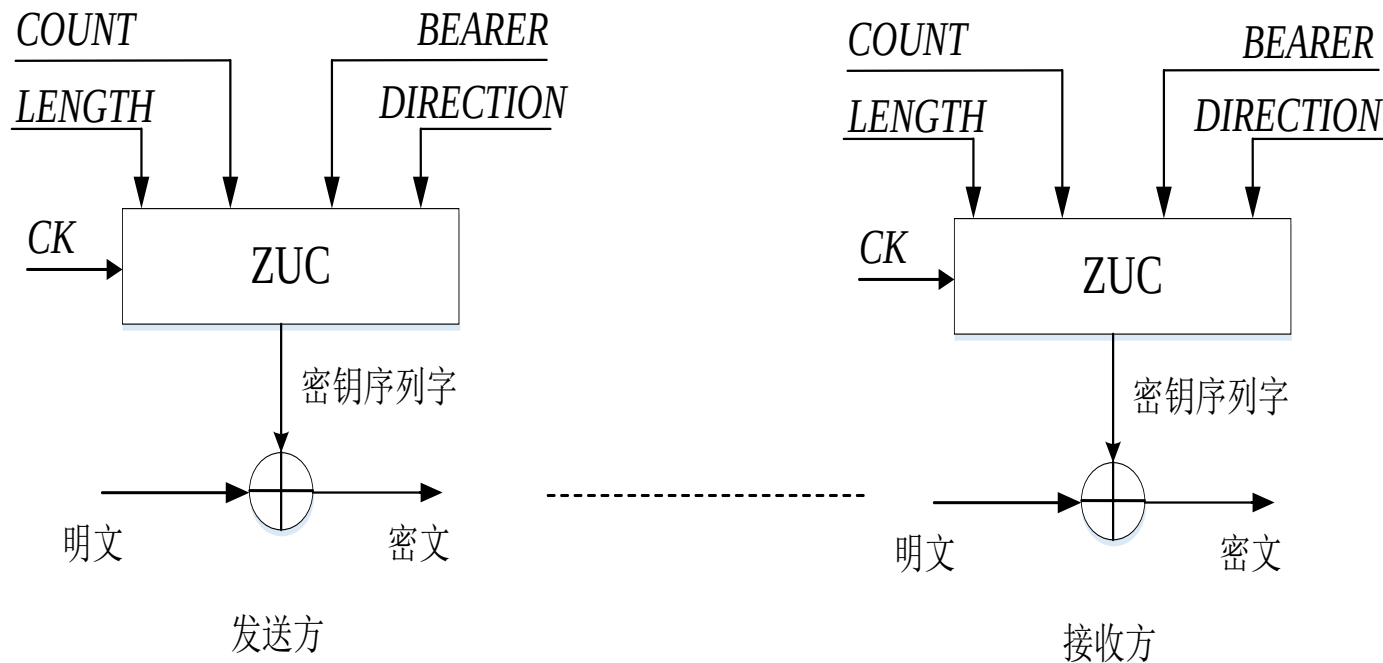


图3-26 基于祖冲之密码的机密性算法128-EEA3

3.8.4 祖冲之密码的机密性算法 128-EEA3

密钥流的产生

设消息长为 $LENGTH$ 比特，由初始化算法得到的初始密钥和初始向量，调用ZUC密码产生 L 个字（每个32比特长）的密钥，其中 $L = \lceil LENGTH / 32 \rceil$

$$L = \lceil LENGTH / 32 \rceil$$

将生成的密钥流用比特串表示为 $z[0], z[1], \dots, z[L-1]$ ，其中 $z[0]$ 为ZUC算法生成的第一个密钥字的最高位比特， $z[L-1]$ 为最低位比特，其他以此类推。



3.8.4 祖冲之密码的机密性算法 128-EEA3

加解密

密钥流产生之后，数据的加解密就十分简单了。
设长度为 $LENGTH$ 的输入消息的比特流为

$$M = M[0] \parallel M[1] \parallel M[2] \parallel \cdots \parallel M[LENGTH-1]$$

则输出的密文比特流为

$$C = C[0] \parallel C[1] \parallel C[2] \parallel \cdots \parallel C[LENGTH-1]$$

其中 $C[i] = M[i] \oplus z[i]$, $i = 0, 1, 2, \cdots, LENGTH-1$





第2单元 分组密码

提纲

CONTENTS

2.1 分组密码概述

2.2 分组密码分析

2.3 DES

2.4 AES

2.5 SM4

2.6 祖冲之密码

2.7 其他分组密码

2.8 分组密码工作模式

王华

2.7 其他分组密码



2.7.1 IDEA

- IDEA (International Data Encryption Algorithm, 国际数据加密算法) 由来学嘉(X. J. Lai)和J. L. Massey (导师) 提出。
- 第一版于1990年公布, 当时称为PES (Proposed Encryption Standard, 建议加密标准)。
- 1991年, 在Biham和Shamir提出差分密码分析之后, 设计者推出了改进算法IPES, 即改进型建议加密标准。
- 1992年, 设计者又将IPES改名为IDEA。这是近年来提出的各种分组密码中一个很成功的方案, 已在PGP中采用。

王华



1、算法参数

- 算法中明文和密文分组长度都是64比特，密钥长128比特。
- 其设计原理可从强度和实现两方面考虑。

王华



2、设计原理-密码强度

算法的强度主要是通过有效的混淆和扩散特性而得以保证。混淆是通过使用以下三种运算而获得，三种运算都有两个16比特的输入和一个16比特的输出：

- 逐比特异或，表示为 $\oplus$ ；
- 模 2^{16} （即65536）整数加法，表示为 $\boxed{+}$ ，其输入和输出作为16位无符号整数处理。
- 模 $2^{16} + 1$ （即65537）整数乘法，表示为 $\odot$ ，其输入、输出中除16位全为0作为 2^{16} 处理外，其余都作为16位无符号整数处理。

王卫华



2、设计原理-密码强度

- IDEA中的运算都是两字节16位的运算。
- $\oplus$ ，按位异或，是 F_2 中的加法运算。
- $\boxplus$ ，通常加法再模 2^{16} ，是 Z_{65536} 上的加法运算，即正常的加法然后取低16位。
- $\odot$ ，通常乘法再模 $(2^{16} + 1)$ ，是 F_{65537}^* 上的乘法运算。当遇到全0时，将其定义为 2^{16} 。即：
 $(0, 0, \dots, 0, 0) \odot (1, 0, \dots, 0, 0) = (1, 0, \dots, 0, 1)$ 。
因为 $2^{16} 2^{15} = 2^{15} + 1$ 。

王卫华



2、设计原理-密码强度

表3-6给出了操作数为2比特长时三种运算的运算表。

表3-6 IDEA中的三种运算（操作数为2比特长）

X	Y	$X \boxplus Y$	$X \odot Y$	$X \oplus Y$
0 00	0 00	0 00	1 01	0 00
0 00	1 01	1 01	0 00	1 01
0 00	2 10	2 10	3 11	2 10
0 00	3 11	3 11	2 10	3 11
1 01	0 00	1 01	0 00	1 01
1 01	1 01	2 10	1 01	0 00
1 01	2 10	3 11	2 10	3 11
1 01	3 11	0 00	3 11	2 10
2 10	0 00	2 10	3 11	2 10
.....				

王立华



2、设计原理-密码强度

在以下意义下，三种运算是不兼容的：

- 三个运算中任意两个都不满足分配律，例如：

$$a \oplus (b \odot c) \neq (a \oplus b) \odot (a \oplus c)$$

- 三个运算中任意两个都不满足结合律，例如：

$$a \oplus (b \oplus c) \neq (a \oplus b) \oplus c$$

三种运算结合起来使用可对算法的输入提供复杂的变换，从而使得对IDEA的密码分析比对仅使用异或运算的DES更为困难。

王华

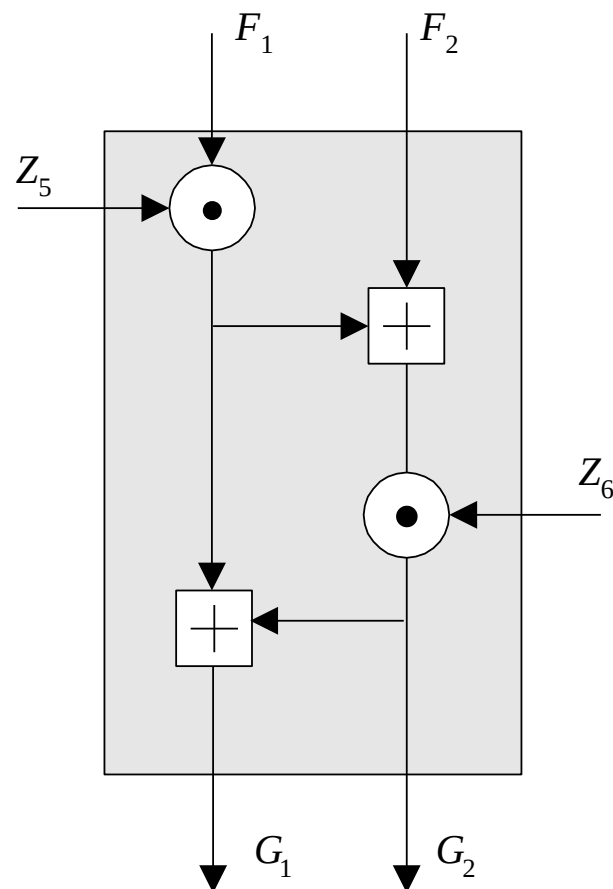


2、设计原理-密码强度

算法中扩散是由称作乘加 (MA- Multiplication/ Addition) 结构 (看图3-14) 的基本单元实现的。该结构的输入是两个16比特的子段和两个16比特的子密钥, 输出也为两个16比特的子段。这一结构在算法中重复使用了8次, 获得了非常有效的扩散效果。

2022/4/9

2、设计原理-密码强度



2022/4/9



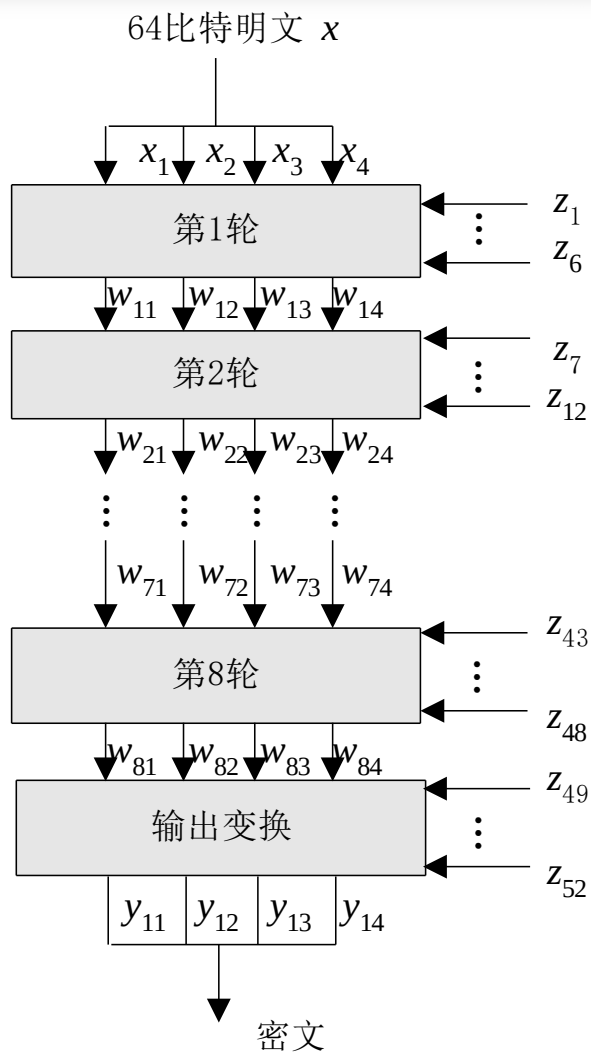
3、加密过程

- 加密过程（如图3-15所示）由连续的8轮迭代和一个输出变换组成，算法将64比特的明文分组分成4个16比特的子段，每轮迭代以4个16比特的子段作为输入，输出也为4个16比特的子段。
- 最后的输出变换也产生4个16比特的子段，链接起来后形成64比特的密文分组。
- 每轮迭代还需使用6个16比特的子密钥，最后的输出变换需使用4个16比特的子密钥，所以子密钥总数为52。
- 图的右部分表示由初始的128比特密钥产生52个子密钥的子密钥产生器。

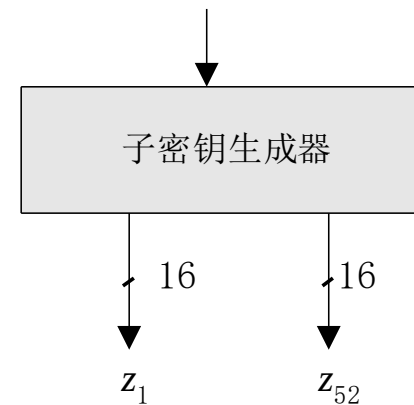
王立华



3、加密过程

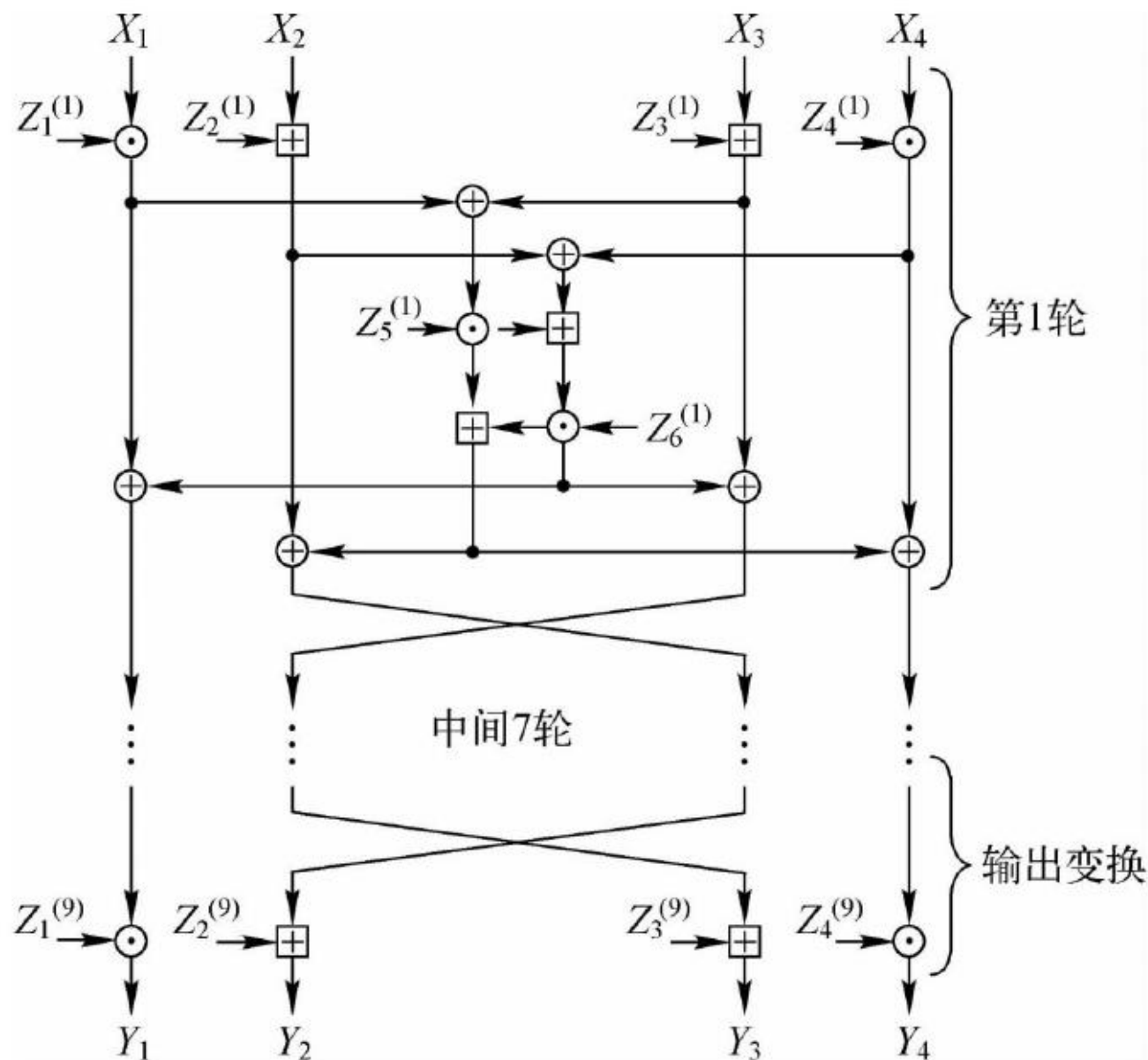


128比特密钥 z



2022/4/9

3、加密过程





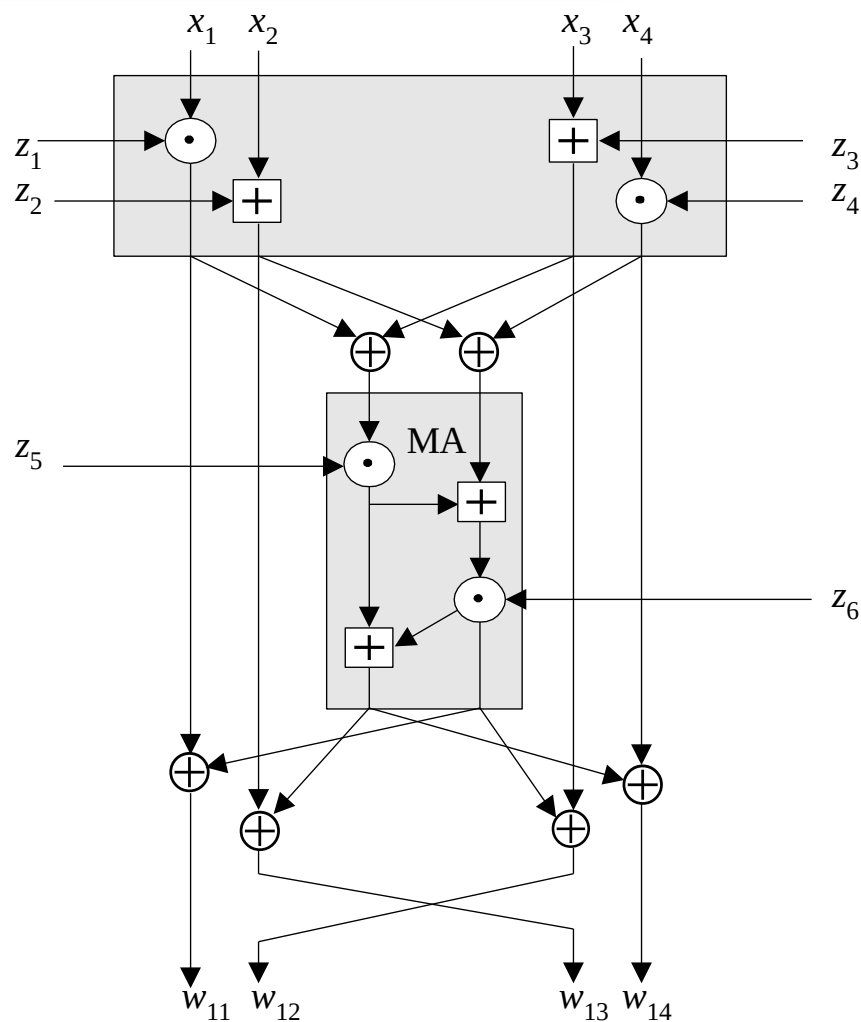
3、加密过程：轮结构

- 图3-16是IDEA第一轮的结构示意图，以后各轮也都是这种结构，但所用的子密钥和轮输入不同。
- 从结构图可见，IDEA不是传统的Feistel密码结构。每轮开始时有一个变换，该变换的输入是4个子段和4个子密钥，变换中的运算是两个乘法和两个加法，输出的4个子段经过异或运算形成了两个16比特的子段作为MA结构的输入。
- MA结构也有两个输入的子密钥，输出是两个16比特的子段。

王立华



3、加密过程：轮结构



2022/4/9



3、加密过程：轮结构

最后，变换的4个输出子段和MA结构的两个输出子段经过异或运算产生这一轮的4个输出子段。注意，由 产生的输出子段和由 产生的输出子段交换位置后形成 W_{12} 和 W_{13} ，目的在于进一步增加混淆效果，使得算法更易抵抗差分密码分析。

算法的第九步是一个输出变换，如图3-17所示。它的结构和每一轮开始的变换结构一样，不同之处在于输出变换的第二个和第三个输入首先交换了位置，目的在于撤消第八轮输出中两个子段的交换。还需注意，第九步仅需4个子密钥而前面8轮中每轮需要6个子密钥。

王立华



3、加密过程：子密钥的产生

IDEA 使用 **52** 个 **16** 位的子密钥，分为九组：

$Z_1^{(1)}, Z_1^{(2)}, \dots, Z_1^{(6)}; Z_2^{(1)}, Z_2^{(2)}, \dots, Z_2^{(6)}; \dots;$

$Z_8^{(1)}, Z_8^{(2)}, \dots, Z_8^{(6)}; Z_1^{(9)}, Z_2^{(9)}, Z_3^{(9)}, Z_4^{(9)}.$

第 **1** 至 **8** 组，每组 **6** 个 **16** 位子密钥。第 **9** 组只有 **4** 个 **16** 位子密钥。最初的 **128** 位密钥分配给 $Z_1^{(1)}, \dots, Z_2^{(2)}$ ，然后将密钥整体左移 **25** 位，再分配给后面的 **8** 个，即赋给 $Z_2^{(3)}, \dots, Z_3^{(4)}$ 。再继续左移 **25** 位，赋给接下来的 **8** 个，依此类推。

王华



3、加密过程：子密钥的产生

加密过程中52个16比特的子密钥是由128比特的加密密钥按如下方式产生的：前8个子密钥 $z_1, z_2, \dots, z_8$ 直接从加密密钥中取，即 z_1 取前16比特（最高有效位）， z_2 取下面的16比特，等等。然后加密密钥循环左移25位，再取下面8个子密钥 $z_9, z_{10}, \dots, z_{16}$ ，取法与 $z_1, z_2, \dots, z_8$ 的取法相同。这一过程重复下去，直到52子密钥都被产生为止。

王华



4、解密过程

解密过程和加密过程基本相同，但子密钥的选取不同。解密子密钥 $U_1, U_2, \dots, U_{52}$ 是由加密子密钥按如下方式得到（将加密过程最后一步的输出变换当作第九轮）：

第 i ($i = 1, \dots, 9$) 轮解密的前4个子密钥由加密过程第 $(10 - i)$ 轮的前4个子密钥得出：其中第1和第4个解密子密钥取为相应的第1和第4个加密子密钥的模 $2^{16} + 1$ 乘法逆元，第2和第3个子密钥的取法为：当轮数 $i = 2, \dots, 8$ 时，取为相应的第3个和第2个加密子密钥的模 2^{16} 加法逆元。 $i = 1$ 和 9 时，取为相应的第2个和第3个加密子密钥的模 2^{16} 加法逆元。

2022/4/9



4、解密过程

第 i ($i = 1, \dots, 8$) 轮解密的后两个子密钥等于加密过程第 $(9-i)$ 轮的后两个子密钥。

表3-7是对以上关系的总结。其中 Z_j 的模 $2^{16}+1$ 乘法逆元为 Z_j^{-1} ，满足：

$$Z_j \odot Z_j^{-1} = 1 \bmod(2^{16} + 1)$$

因 $2^{16}+1$ 是一素数，所以每一个不大于 2^{16} 的非0整数都有一个惟一的模 $2^{16}+1$ 乘法逆元。

Z_j 的模 2^{16} 加法逆元为 $-Z_j$ ，满足：

$$-Z_j \boxplus Z_j = 0 \bmod(2^{16})$$

王卫华



4、解密过程

表3-7 IDEA加密、解密子密钥表

轮数	加密子密钥	解密子密钥	
1	$Z_1, Z_2, Z_3, Z_4, Z_5, Z_6$	$U_1, U_2, U_3, U_4, U_5, U_6$	$Z_{49}^{-1}, -Z_{50}, -Z_{51}, Z_{52}^{-1}, Z_{47}, Z_{48}$
2	$Z_7, Z_8, Z_9, Z_{10}, Z_{11}, Z_{12}$	$U_7, U_8, U_9, U_{10}, U_{11}, U_{12}$	$Z_{43}^{-1}, -Z_{45}, -Z_{44}, Z_{46}^{-1}, Z_{41}, Z_{42}$
3	$Z_{13}, Z_{14}, Z_{15}, Z_{16}, Z_{17}, Z_{18}$	$U_{13}, U_{14}, U_{15}, U_{16}, U_{17}, U_{18}$	$Z_{37}^{-1}, -Z_{39}, -Z_{38}, Z_{40}^{-1}, Z_{35}, Z_{36}$
4	$Z_{19}, Z_{20}, Z_{21}, Z_{22}, Z_{23}, Z_{24}$	$U_{19}, U_{20}, U_{21}, U_{22}, U_{23}, U_{24}$	$Z_{31}^{-1}, -Z_{33}, -Z_{32}, Z_{34}^{-1}, Z_{29}, Z_{30}$
5	$Z_{25}, Z_{26}, Z_{27}, Z_{28}, Z_{29}, Z_{30}$	$U_{25}, U_{26}, U_{27}, U_{28}, U_{29}, U_{30}$	$Z_{25}^{-1}, -Z_{27}, -Z_{26}, Z_{28}^{-1}, Z_{23}, Z_{24}$
6	$Z_{31}, Z_{32}, Z_{33}, Z_{34}, Z_{35}, Z_{36}$	$U_{31}, U_{32}, U_{33}, U_{34}, U_{35}, U_{36}$	$Z_{19}^{-1}, -Z_{21}, -Z_{20}, Z_{22}^{-1}, Z_{17}, Z_{18}$
7	$Z_{37}, Z_{38}, Z_{39}, Z_{40}, Z_{41}, Z_{42}$	$U_{37}, U_{38}, U_{39}, U_{40}, U_{41}, U_{42}$	$Z_{13}^{-1}, -Z_{15}, -Z_{14}, Z_{16}^{-1}, Z_{11}, Z_{12}$
8	$Z_{43}, Z_{44}, Z_{45}, Z_{46}, Z_{47}, Z_{48}$	$U_{43}, U_{44}, U_{45}, U_{46}, U_{47}, U_{48}$	$Z_7^{-1}, -Z_9, -Z_8, Z_{10}^{-1}, Z_5, Z_6$
输出变换	$Z_{49}, Z_{50}, Z_{51}, Z_{52}$	$U_{49}, U_{50}, U_{51}, U_{52}$	$Z_1^{-1}, -Z_2, -Z_3, Z_4^{-1}$

2022/4/9



4、解密过程

下面验证解密过程的确得到正确的结果。图3-18中左边为加密过程，由上至下，右边为解密过程，由下至上。将每一轮进一步分为两步，第一步是变换，其余部分作为第二步，称为子加密。

从下往上考虑。对加密过程的最后一个输出变换，以下关系成立：

$$Y_1 = W_{81} \odot Z_{49}$$

$$Y_3 = W_{82} \boxed{+} Z_{51}$$

$$Y_2 = W_{83} \boxed{+} Z_{50}$$

$$Y_4 = W_{84} \odot Z_{52}$$

王华



4、解密过程

解密过程中第1轮的第1步产生以下关系：

$$\begin{aligned} J_{11} &= Y_1 \odot U_1 & J_{12} &= Y_2 \boxed{+} U_2 \\ J_{13} &= Y_3 \boxed{+} U_3 & J_{14} &= Y_4 \odot U_4 \end{aligned}$$

将解密子密钥由加密子密钥表达并将 代入以下关系，有：

$$\begin{aligned} J_{11} &= Y_1 \odot Z_{49}^{-1} = W_{81} \odot Z_{49} \odot Z_{49}^{-1} = W_{81} \\ J_{12} &= Y_2 \boxplus -Z_{50} = W_{83} \boxplus Z_{50} \boxplus -Z_{50} = W_{83} \\ J_{13} &= Y_3 \boxplus -Z_{51} = W_{82} \boxplus Z_{51} \boxplus -Z_{51} = W_{82} \\ J_{14} &= Y_4 \odot Z_{52}^{-1} = W_{84} \odot Z_{52} \odot Z_{52}^{-1} = W_{84} \end{aligned}$$

2022/4/9



4、解密过程

可见解密过程第1轮第1步的输出等于加密过程最后一步输入中第2个子段和第3个子段交换后的值。从图3-16，可得以下关系：

$$W_{81}=I_{81} \oplus MA_R (I_{81} \oplus I_{83}, I_{82} \oplus I_{84})$$

$$W_{82}=I_{83} \oplus MA_R (I_{81} \oplus I_{83}, I_{82} \oplus I_{84})$$

$$W_{83}=I_{82} \oplus MA_L (I_{81} \oplus I_{83}, I_{82} \oplus I_{84})$$

$$W_{84}=I_{84} \oplus MA_L (I_{81} \oplus I_{83}, I_{82} \oplus I_{84})$$

2022/4/9



4、解密过程

解密过程

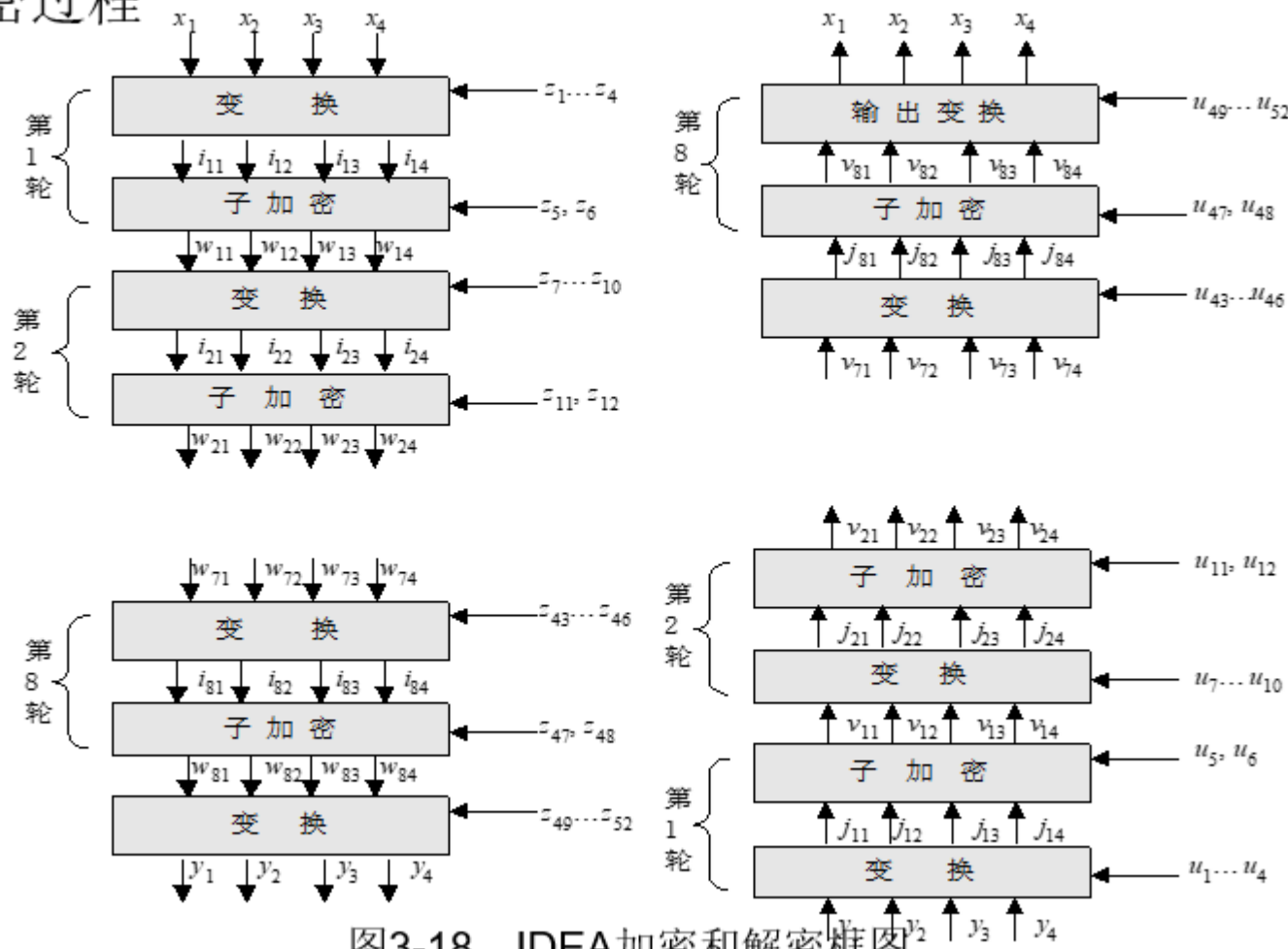


图3-18 IDEA加密和解密框图

2022/4/9



4、解密过程

其中 $MA_R(X,Y)$ 是MA结构输入为 X 和 Y 时的右边输出
 $MA_L(X,Y)$ 是左边输出。则：

$$\begin{aligned}
 V_{11} &= J_{11} + MA_R(J_{11} + J_{13}, J_{12} + J_{14}) = W_{81} + MA_R(W_{81} + W_{82}, W_{83} + W_{84}) \\
 &= I_{81} + MA_R(I_{81} + I_{83}, I_{82} + I_{84}) + MA_R[I_{81} + MA_R(I_{81} + I_{83}, I_{82} + I_{84}) + I_{83} + MA_R \\
 &\quad (I_{81} + I_{83}, I_{82} + I_{84}), I_{82} + MA_L(I_{81} + I_{83}, I_{82} + I_{84}) + I_{84} + MA_L(I_{81} + I_{83}, \\
 &\quad I_{82} + I_{84})] \\
 &= I_{81} + MA_R(I_{81} + I_{83}, I_{82} + I_{84}) + MA_R(I_{81} + I_{83}, I_{82} + I_{84}) = I_{81}
 \end{aligned}$$

类似地，可有：

$$V_{12} = I_{83}$$

$$V_{13} = I_{82}$$

$$V_{14} = I_{84}$$

2022/4/9



4、解密过程

所以解密过程第1轮第2步的输出等于加密过程倒数第2步输入中第2个子段和第3个子段交换后的值。

同理可证图3-18中每步都有上述类似关系，这种关系一直到

$$V_{81}=I_{11} \quad V_{82}=I_{13} \quad V_{84}=I_{14} \quad V_{83}=I_{12}$$

即除第2个子段和第3个子段交换位置外，解密过程的输出变换与加密过程第1轮第1步的变换完全相同。

所以最后得出整个解密过程的输出等于整个加密过程的输入。

王卫华



5、算法安全性

- 算法中的 $\odot$ 和田不满足分配律：

$$a \odot (b \boxplus c) \neq (a \odot b) \boxplus (a \odot c)$$

$$a \boxplus (b \odot c) \neq (a \boxplus b) \odot (a \boxplus c)$$

- 算法中的 $\odot$ 和田具有很好的扩散和混淆效果，且 $\odot$ 运算是非线性的，具有较好的抗差分分析作用。
- 加乘器中的交换也有抗差分分析作用。
- **IDEA**针对现有的分析方法是安全的。

吴志华



6、IDEA小结

- 算法结构清晰简洁。
- 算法的三种运算都有清晰的代数结构，易于给出其可接受的安全性指标。
- 算法的密钥长度目前是安全的。
- 算法的软硬件实现都比较简单、快捷。

王卫华